Continuous Deployment Orchestration *Using* Rundeck

Continuous integration (CI) and continuous deployment (CD) are fast becoming accepted best practices in the software and application development world. Together, these strategies are known as continuous deployment. This article explores continuous deployment practices and focuses on Rundeck, a CD orchestration tool.

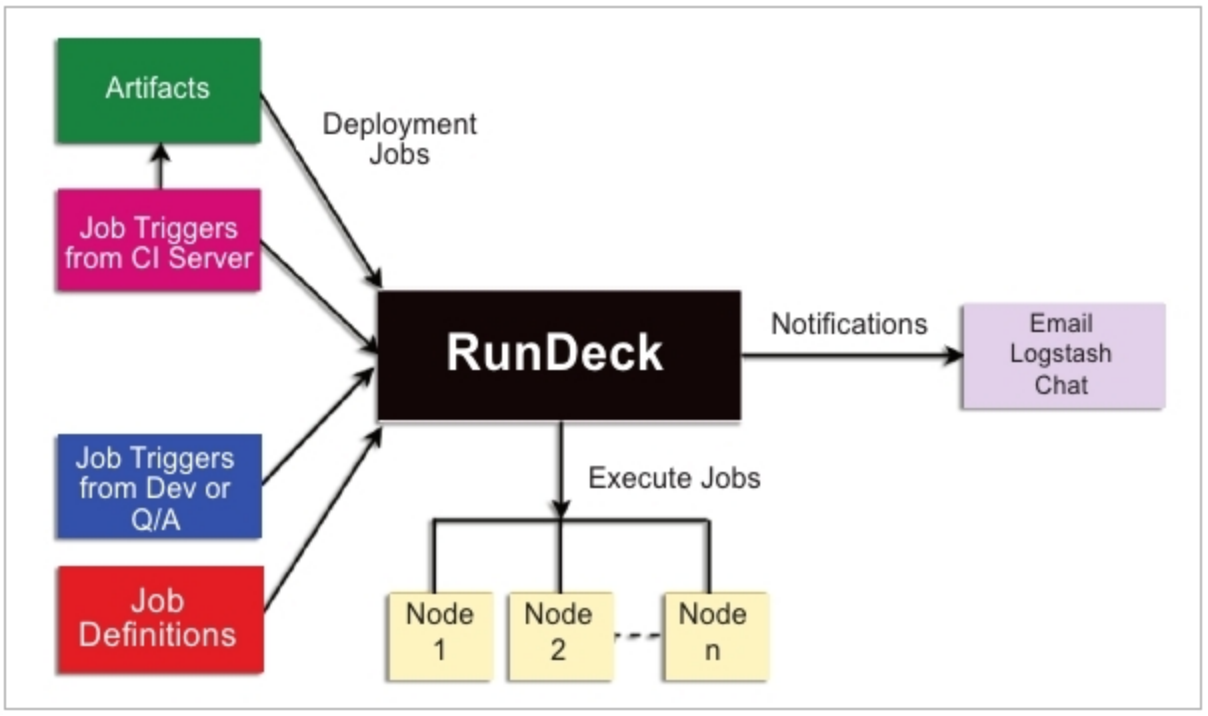

Figure 1: Deployment automation using Rundeck

In this age, when technology is advancing quickly and becoming more widespread, minimising the time to market is very important for electronics products and services. Apart from a shorter time to market, the software of the product/service has to be constantly updated with the latest security patches, bug fixes and new features, to keep attracting new users as well as to maintain the existing user base. In addition, all the releases of the software have to be thoroughly tested and must be stable. All of this can be achieved by implementing continuous delivery and continuous deployment in the software delivery pipeline. Let us have a closer look at continuous deployment practices and the need for these in software development organisations. In this context, we will explore a service orchestration tool called Rundeck, and look at its features as well as installation procedure on Linux machines.

Introducing continuous practices

The various stages of the software development life cycle (SDLC), according to the Agile methodology, are meet, plan, design, develop, test, release and deploy, as shown in Figure 2. In organisations where there are hundreds of ongoing projects with more than one team working on the same project, there is a huge possibility of code conflicts, manual errors and configuration gaps,

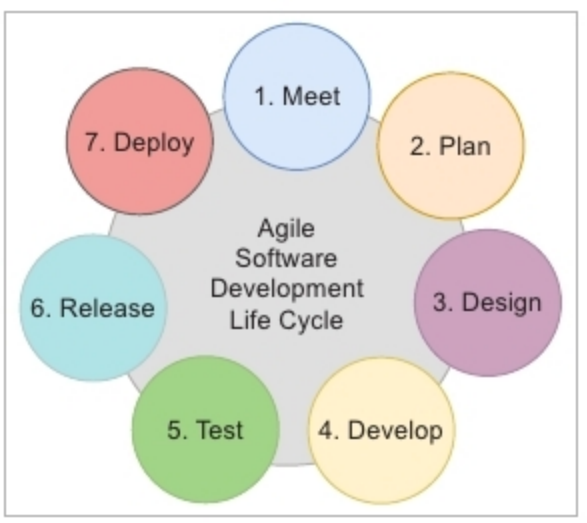

Figure 2: Agile software development life cycle

which result in delayed releases.

Building an automated CI/CD pipeline using tools for the build, testing, deployment and monitoring stages is an effective method to overcome these problems. Figure 3 shows a CI/CD pipeline of software delivery. The next section gives a brief explanation of CI and CD.

Continuous integration combines the source code from multiple developers working on the same project, and automatically builds the code. This is usually done by monitoring the version control manager for any new changes in the source code. If there are any changes or a code commit, the CI tool automatically builds the code and tests the application. CI is the first step towards continuous deployment. Some examples of CI tools are Jenkins, Travis and Cruise Control.

After CI, the next phase in the CI/CD pipeline is continuous delivery. The code/software is always made ready to go into production even after making new changes. This is continuous delivery (CDE), which makes every change production ready. Automating the process of deployment into production is called continuous deployment (CD), which cannot be achieved without having continuous delivery. CDE and CD are shown in Figure 4, where the version control manager is Git and the continuous integration tool is Jenkins.

In a competitive market that includes mobile, online banking and e-commerce applications, frequent updates to apps can give an edge to organisations in terms of customer satisfaction. For this, the companies